# communitycore 插件深度审查报告

> 版本：1.0.0 | 类型：后端插件 | 代码量：~1,800 行
>
> 审查日期：2026-04-03 | 更新：**2026-04-03（P0 + P1 优化已落地）**

## 综合评分

| 维度 | 审查时 | 优化后 | 简评 |
|---|---|---|---|
| 功能性 | 8/10 | 9/10 | `getArticleDetail` 现已包含文章正文 |
| 安全性 | 2/10 | 7/10 | JWT 中间件 + 所有权校验已落地；`queryArticles` 裸 SQL 仍存在 |
| 架构设计 | 6/10 | 8/10 | Repository 职责已收归，直调 DatabaseService 已消除 |
| 接口一致性 | 4/10 | 5/10 | 全局错误中间件统一了 500 响应；400 格式仍为 JSON |
| 错误处理 | 3/10 | 7/10 | 全局异常捕获中间件已加入，所有权校验返回语义化 403/404 |
| 可测试性 | 7/10 | 8/10 | 70 个测试通过（+3 新增），覆盖所有权校验场景 |
| 总评 | **5/10** | **7/10** | 安全基线与架构分层明显改善，遗留问题已列入 P2/P3 |

# 一、功能性分析

## 已实现功能（✅ 覆盖全面）

| 模块 | 路由数 | 说明 |
| --- | --- | --- |
| 文章 | 5 | createArticle、modifyArticle、deleteArticle、getArticles、getArticleDetail |
| 评论 | 4 | addComment、deleteComment、modifyComment、getComments |
| 点赞 | 6 | 文章 / 评论各自的 like / cancelLike / getLikes |
| 媒体 | 3 | addMedia、deleteMedia、getMedias |
| 分类 | 1 | getCategory |
| 关注 | 3 | followUser、unfollowUser、getFollow |
| 阅读 | 2 | readArticle、getArticleReads |
| 审核管理 | 10 | 黑名单 / 屏蔽分类 / 举报 / 不喜欢 各两到三个接口 |

## 功能缺陷

| 缺陷 | 严重度 | 状态 | 说明 |
| --- | --- | --- | --- |
| `getArticleDetail` 不含文章正文 | 🟠 高 | ✅ 已修复 | proto 新增 `article = 7` 字段，handler 调用 `getArticleById` 返回完整文章 |
| `getArticles` 无法按分类 ID 查询 | 🟡 中 | ⏳ P3 | proto 中有 `CategoryID` 字段，实现侧未使用 |
| Follow 流的帖子不包含作者信息 | 🟡 中 | ⏳ P3 | `queryArticles` 只查 Article 表，用户名需客户端二次请求 |

## 二、安全性批判

### ✅ 已修复：所有路由接入 JWT 鉴权中间件

新增 `lib/routes/auth_middleware.dart`，在 `communityErrorMiddleware` + `communityAuthMiddleware` 组成的 Pipeline 中统一鉴权：

```
// community_routes.dart
void setupCommunityRoutes({required String jwtSecret, CommunityNotifier?
notifier}) {
  final pipeline = const Pipeline()
      .addMiddleware(communityErrorMiddleware())
      .addMiddleware(communityAuthMiddleware(jwtSecret));
  // 所有路由均通过 pipeline.addHandler 包裹
}
```

- 无 Bearer token → 401
- token 过期 → 401（`JWTExpiredException` 单独处理）
- token 有效，userId 从 `jwt.jwtId` 提取并注入 `request.context`
- 后续 handler 通过 `getCallerUserId(req)` 取得已验证的 userId

> 调用方变更：`setupCommunityRoutes()` 现在必须传入 `jwtSecret`：
>
> ```dart
> router.setupCommunityRoutes(jwtSecret: env['JWTSECRET']!);
> ```

### ✅ 已修复：写操作所有权校验

`_modifyArticle` 和 `_deleteArticle` 均加入：

```
final callerUserId = getCallerUserId(req);
final existingArticle = await repo.getArticleById(caReq.articleID);
if (existingArticle == null) return Response.notFound(null);          // 404
if (existingArticle[ArticleTable.authorID] != callerUserId) return
forbidden(); // 403
```

---

### 🟠 高风险问题（未修复）：`queryArticles` 接受裸 SQL 字符串

```
// community_repository.dart
Future<List<Map<String, dynamic>>> queryArticles(String sql) async {
  final results = await _db.query(sql);  // 直接执行，无参数化
  return results.map((r) => r.fields).toList();
}
```

批判：调用方将用户输入的 `caReq.userID`（虽然是 int32，较安全）直接插入 SQL 字符串。但这个接口本身的设计让路由层完全控制 SQL 构造，未来扩展时极易引入 SQL 注入：

```
// article_routes.dart 中的调用
```

```
final sql = '''
SELECT ... FROM articles WHERE ...
AND authorID NOT IN (SELECT ... WHERE userId = ${caReq.userID})
ORDER BY publishTime DESC
''';
list = await repo.queryArticles(sql);  // 用户控制的 int 已插入字符串
```

改良方案：将各类查询封装为带参数的 Repository 方法，彻底消除路由层构造 SQL 的场景。

---

## 三、架构设计批判

### ✅ 已修复：`_getArticles` 直调 `DatabaseService` 已消除

`myPublished` / `myDraft` 两个分支的所有直调全部替换为 Repository 方法：

```
// community_repository.dart 新增
Future<List<Map<String, dynamic>>> getPublishedArticlesByAuthor(int
authorID, {int? privacy});
Future<List<Map<String, dynamic>>> getDraftArticlesByAuthor(int authorID,
{int? privacy});
```

路由层不再持有任何直接数据库连接，`FakeCommunityRepository` 可完整拦截所有查询路径。

### 问题 2（未修复）：`queryArticles(String sql)` 设计反模式

Repository 接口应该定义**业务语义方法**，而非作为 SQL 透传通道。当前设计等价于路由层直接操作数据库，Repository 抽象失效。

### 问题 3（未修复）：`cagegory` 拼写错误固化在全栈

```
// community_fields.dart
static const String cagegory = 'Cagegory'; // 与数据库一致（含拼写）
```

正确拼写应为 `category`，`Cagegory` 是拼写错误。注释"与数据库一致"表明数据库列名也是错的，整个错误已被冻结在数据库 schema 和常量中。不修复不影响运行，但会持续扩散到新代码。

### 问题 4（未修复）：`CommunityNotifier` 的扩展点不透明

```
void _sendSystemMsg(String type, String? rid, Map<String, dynamic> data) {
  // 由宿主应用覆盖此方法以发送系统消息
}
```

`_sendSystemMsg` 是空实现，通过子类覆盖来扩展通知能力。但：

- 没有任何接口/抽象类声明此扩展点

- `CommunityNotifier` 是具体类而非抽象类，`setupCommunityRoutes()` 直接 `new CommunityNotifier()`
- 宿主应用必须知道这个约定，否则通知永远静默丢弃

改良方案：

```
abstract class CommunityNotifier {
  void notifyArticleStatusChanged(...);
  void notifyArticleReported(...);
  // 强制宿主实现
}

// 默认空实现作为可选覆盖
class NoOpCommunityNotifier implements CommunityNotifier {
  @override void notifyArticleStatusChanged(...) {}
  @override void notifyArticleReported(...) {}
}
```

---

## 四、接口一致性批判

### ✅ 部分修复：500 响应已统一

全局 `communityErrorMiddleware` 将所有未捕获异常统一返回带 body 的 JSON：

```
{"code": 500, "msg": "Internal server error"}
```

### 仍存在问题：正常响应与错误响应格式混搭

| 场景 | 响应格式 | 状态码 |
|---|---|---|
| 正常响应 | protobuf 二进制 | 200 |
| 参数错误 | JSON `{"code":2,"msg":"..."}` | 400 |
| Content-Type 错误 | JSON `{"code":1,"msg":"..."}` | 400 |
| 所有权/鉴权错误 | JSON `{"code":403/401,"msg":"..."}` | 403 / 401 |
| 服务端错误 | JSON `{"code":500,"msg":"..."}` | 500 ✅ |

客户端对正常 200 响应仍需用 protobuf 解析，对所有非 200 响应需用 JSON 解析，两套策略共存。

---

## 五、错误处理批判

### ✅ 已修复：全局异常捕获中间件

`communityErrorMiddleware` 捕获所有未处理异常，记录日志并返回标准化 JSON 500 响应，不再泄露内部栈信息。

### ✅ 已修复：语义化错误状态码

- 文章不存在 → `404 Not Found`（原为空 body 500）
- 非所有者操作 → `403 Forbidden`（原无此校验）
- token 无效/缺失 → `401 Unauthorized`（原无此校验）

---

## 六、改良建议总结

| 优先级 | 改良项 | 状态 |
|---|---|---|
| P0 | 接入 JWT 鉴权中间件，所有写操作验证 token userId | ✅ 已完成 |
| P0 | 写操作加所有权校验（deleteArticle/modifyArticle 验证 authorID） | ✅ 已完成 |
| P1 | 将 `myPublished` / `myDraft` 查询移入 Repository | ✅ 已完成 |
| P1 | 添加全局错误处理中间件 | ✅ 已完成 |
| P1 | `getArticleDetail` 在响应中包含文章正文 | ✅ 已完成 |
| P1 | 统一错误响应格式（消除 500 裸空 body） | ✅ 已完成（500 统一；400 格式维持现状） |
| P2 | 将 `CommunityNotifier` 改为抽象接口 | ⏳ 待处理 |
| P2 | 淘汰 `queryArticles(String sql)`，用类型化方法替代 | ⏳ 待处理 |
| P3 | 修复 `cagegory` 拼写错误（需同步 DB Migration） | ⏳ 待处理 |
| P3 | proto 枚举的分类字段在路由中实现 | ⏳ 待处理 |